从函数看实现：一条新闻如何完成热 → 温迁移

先看 process_job() 主流程，再看 activate() 的状态切换；下方 A-G 对应真实文件与当前行号。

推荐断点：A:180 scan_page → B:247 request_migration → A:143 process_job → B:357 activate

01 谁判断层级，谁创建任务？

C:96 classify_tier()

发布时间 + now + 回热保持期
输出 HOT / WARM / COLD

→

A:180 scan_page()

分页读台账，使用冻结策略
目标层变化 → request_migration

→

D:48 dispatch_once()

PG pending/retry → Temporal
Workflow → Activity → process_job

02 process_job()：顺着这五步读

A:93
_records()
切片 + Embedding
先保存向量快照

→

A:162
index.upsert()
分批写入目标层
此时尚不可查询

→

A:163
verify_records()
完整元数据校验
失败不切换

→

B:357
activate()
核对 generation
修改 active_job / tier

→

A:137
reconcile()
只删除 obsolete 清单
保留当前有效投影

03 看字段变化，比只看状态名称更容易理解

阶段	revision	generation	desired	active	active job	current job
迁移前	7	11	hot	hot	J11	J11
已创建迁移	7	12	warm	hot	J11	J12
验证并激活	7	12	warm	warm	J12	J12

B:357 activate() 的条件

没有撤稿、任务没有终止失败；
任务 generation = 新闻 generation；
current_job_id = 本次 job_id；
投影已保存。满足后才可切换。

04 为什么旧向量还没删，也不会被返回？

E:16 search()：一次 Embedding → 三层并发 → F:61 单层过滤 / 补搜 → B:434 最终状态校验 → 去重排序。
filter_hits 只接受 active 新闻，且内容版本、有效层、模型版本、chunk_id 都与当前投影一致。耗时输出在 E:60。

调试用例：G:225 迁移保留源层并验证清理；模拟失败恢复：G:255。

A agent/app/knowledge/tier_lifecycle.py
B agent/app/knowledge/tier_lifecycle_store.py
C agent/app/knowledge/tier_lifecycle_config.py
D agent/app/knowledge_lifecycle_worker.py
E agent/app/knowledge/tier_lifecycle_query.py
F agent/app/knowledge/tier_lifecycle_index.py
G agent/tests/test_tier_lifecycle_runtime.py

仅源码定位；行号随代码修改可能变化。依赖：PostgreSQL / Temporal / Embedding / Milvus。